

1 Grep

grep è un comando potente per cercare testo all'interno di file o flussi di output in Bash. È molto utile per trovare stringhe specifiche o pattern usando le espressioni regolari. La sintassi è la seguente:

```
1 grep [opzioni] 'pattern' [file]
```

Dove:

- **pattern**: il testo o la regex che vuoi cercare.
- **file**: il file o i file in cui cercare (se omissso, **grep** legge dall'input standard).

1.1 Opzioni comuni

1. **-i**: Cerca senza distinzione tra maiuscole e minuscole.
2. **-r** o **-R**: Cerca ricorsivamente nelle sottocartelle.
3. **-v**: Mostra le righe che non corrispondono al pattern.
4. **-n**: Mostra il numero di riga accanto al risultato.
5. **-l**: Mostra solo i nomi dei file che contengono il pattern.
6. **-c**: Conta quante volte il pattern appare in ogni file.
7. **-o**: Mostra solo il pattern trovato invece della riga intera.

1.2 Esempi

```
1 grep 'errore' log.txt
```

Questo comando cerca la parola "errore" nel file **log.txt** e restituisce le righe che contengono la parola.

```
1 grep -i 'avviso' log.txt
```

Cerca "avviso" o "Avviso" (o altre variazioni di maiuscole e minuscole) in **log.txt**.

```
1 grep -r 'errore' /var/log
```

Cerca la parola "errore" in tutti i file all'interno della directory **/var/log** e delle sue sottodirectory.

```
1 grep -l 'login' *.log
```

Cerca la parola "login" in tutti i file con estensione **.log** nella directory corrente e mostra solo i nomi dei file che la contengono.

```
1 grep -o '[0-9]\{3\}' log.txt
```

Cerca qualsiasi sequenza di tre cifre in **log.txt** e restituisce solo queste sequenze invece dell'intera riga. In particolare:

[0-9]: rappresenta una cifra singola, quindi può essere un numero tra 0 e 9.

\{3\}: indica che stiamo cercando esattamente tre occorrenze consecutive del pattern precedente.

2 Head/Tail

I comandi `head` e `tail` in Bash sono utili per visualizzare rispettivamente le prime o le ultime righe di un file. Entrambi sono molto pratici per esplorare file di testo senza doverli aprire completamente, specie se sono grandi.

2.1 Head

Il comando `head` mostra le prime righe di un file. Per impostazione predefinita, visualizza le prime 10 righe, ma puoi modificare questo numero. La sintassi è:

```
1 head [opzioni] [file]
```

Dove le opzioni comuni sono:

- `-n N`: Specifica il numero di righe da mostrare. Sostituisci *N* con il numero desiderato.
- `-c N`: Mostra i primi N caratteri del file, anziché le righe.

2.1.1 Esempi

Visualizzare le prime 10 righe di un file

```
1 head file.txt
```

Visualizzare le prime 5 righe di un file

```
1 head -n 5 file.txt
```

Mostrare i primi 20 caratteri di un file

```
1 head -c 20 file.txt
```

2.2 Tail

Il comando `tail` è l'opposto di `head`: visualizza le ultime righe di un file. Anche `tail` mostra 10 righe di default, ma come per `head`, puoi specificare un numero diverso. La sintassi di base è:

```
1 tail [opzioni] [file]
```

Le opzioni comuni sono:

- `-n N`: Specifica il numero di righe da mostrare.
- `-c N`: Mostra gli ultimi N caratteri del file.

2.2.1 Esempi

Visualizzare le ultime 10 righe di un file

```
1 tail file.txt
```

Visualizzare le ultime 3 righe di un file

```
1 tail -n 3 file.txt
```

Mostrare gli ultimi 50 caratteri di un file

```
1 tail -c 50 file.txt
```

2.3 Utilizzo combinato di Head e Tail

Puoi anche combinare `head` e `tail` insieme, ad esempio, per estrarre un insieme specifico di righe all'interno di un file.

Estrarre, ad esempio, le righe da 11 a 20 di un file:

```
1 head -n 20 file.txt | tail -n 10
```

3 Sed

sed è un potente comando per modificare testo nei file o nell'output in tempo reale in Bash. Il nome è l'abbreviazione di "stream editor", e può essere usato per cercare, sostituire, inserire o cancellare testo. La sintassi è:

```
1 sed [opzioni] 'espressione' [file]
```

Dove:

- **espressione**: il comando di modifica che vuoi eseguire. Ad esempio, una delle espressioni più comuni è la sostituzione (**s/**).
- **file**: il file su cui applicare l'espressione (se omissso, **sed** legge dall'input standard).

3.1 Opzioni comuni

1. **-i**: Modifica il file in-place, salvando le modifiche direttamente nel file. Se omissso, **sed** mostrerà solo l'output senza modificare il file originale.
2. **-e**: Permette di specificare più espressioni.
3. **-n**: Sopprime l'output automatico; utile se vuoi controllare manualmente cosa **sed** deve mostrare.
4. **-r**: Usa le espressioni regolari estese, semplificando la scrittura delle regex (ad esempio, evitando il backslash per alcuni caratteri speciali).

3.2 Comandi principali

3.2.1 Sostituzione (s/)

Il comando **s/** sostituisce un pattern con un'altra stringa. La sintassi è:

```
1 sed 's/pattern/sostituzione/' file
```

- **pattern**: la stringa o espressione regolare da cercare.
- **sostituzione**: il testo con cui vuoi sostituire il pattern.
- **g**: (opzionale) indica a **sed** di sostituire tutte le occorrenze del pattern in ogni riga (altrimenti sostituisce solo la prima occorrenza di ciascuna riga). Viene messo alla fine.

Esempi:

Cerca la parola "errore" e la sostituisce con "successo", ma solo per la prima occorrenza in ogni riga di **file.txt**.

```
1 sed 's/errore/successo/' file.txt
```

Sostituire tutte le occorrenze in ogni riga

```
1 sed 's/errore/successo/g' file.txt
```

Sostituzione senza distinzione tra maiuscole e minuscole. Qui [Ee] permette di trovare "Errore" o "errore" e sostituirlo con "successo".

```
1 sed 's/[Ee]rrrore/successo/g' file.txt
```

Sostituire direttamente nel file. Con l'opzione -i, sed modifica file.txt direttamente, sostituendo "vecchio" con "nuovo".

```
1 sed -i 's/vecchio/nuovo/g' file.txt
```

3.2.2 Cancellazione di righe (d)

Il comando d è usato per cancellare righe specifiche. La sintassi è:

```
1 sed 'N d' file
```

N è il numero della riga da cancellare.

Esempi:

Cancellare una riga specifica. Questo comando elimina la terza riga di file.txt.

```
1 sed '3d' file.txt
```

Cancellare righe che contengono una parola specifica. Qui sed elimina tutte le righe di file.txt che contengono la parola "errore".

```
1 sed '/errore/d' file.txt
```

3.3 Inserimento di testo (i\) e (a\)

- i\ : inserisce testo prima della riga specificata.
- a\ : aggiunge testo dopo la riga specificata.

Esempi:

Inserisce "Nuova riga di testo" prima della prima riga che contiene "pattern".

```
1 sed '/pattern/i\Nuova riga di testo' file.txt
```

Aggiunge "Nuova riga" dopo la terza riga di file.txt.

```
1 sed '3a\Nuova riga' file.txt
```

3.4 Stampare righe specifiche (p)

L'opzione p permette di stampare righe selezionate, particolarmente utile con -n per sopprimere l'output automatico e mostrare solo le righe che ti interessano.

Esempi:

Mostrare solo le righe contenenti un pattern. Qui -n disattiva l'output automatico, e p stampa solo le righe che contengono "errore".

```
1 sed -n '/errore/p' file.txt
```

Stampare una specifica riga (ad esempio, la seconda)

```
1 sed -n '2p' file.txt
```

Si possono combinare più comandi `sed`. Ad esempio:

```
1 sed -i -e 's/vecchio/nuovo/g' -e '/errore/d' file.txt
```

In questo caso:

- Sostituisce tutte le occorrenze di "vecchio" con "nuovo".
- Cancella tutte le righe che contengono "errore".

4 Awk

Viene utilizzato principalmente per estrarre e manipolare dati in base a colonne specifiche di testo. La sintassi di base è:

```
1 awk 'comando' [file]
```

Dove:

- **comando**: Una serie di istruzioni che **awk** esegue su ogni riga del file.
- **file**: Il file di input su cui applicare il comando.

4.1 Concetti fondamentali

- **awk** tratta ogni riga di un file come una sequenza di campi separati da uno spazio (o un delimitatore specificato), e ogni riga viene trattata come un record. I campi di ciascun record vengono numerati a partire da **\$1** (primo campo), **\$2** (secondo campo), e così via. **\$0** rappresenta l'intera riga.
- **FS** (Field Separator): Variabile che definisce il separatore tra i campi (di default è uno spazio o una tabulazione).
- **OFS** (Output Field Separator): Variabile che definisce il separatore da usare per i campi nell'output (di default è uno spazio).

4.2 Operazioni comuni

4.2.1 Stampa di una Colonna Specifica

Se hai un file con più colonne, puoi usare **awk** per estrarre una colonna specifica. Ecco un esempio:

```
1 awk '{print $1}' file.txt
```

Questo comando stampa solo il primo campo (colonna) di ogni riga del file **file.txt**.

```
1 awk '{print $1, $3}' file.txt
```

Questo comando stampa la prima e la terza colonna di ogni riga.

```
1 awk '{print $0}' file.txt
```

\$0 rappresenta l'intera riga. Quindi questo comando stampa ogni riga del file, proprio come se fosse il contenuto originale.

4.2.2 Modifica del Separatore di Campi

Se i dati sono separati da un delimitatore diverso da uno spazio (come una virgola o un punto e virgola), puoi cambiare il separatore dei campi con **-F**:

```
1 awk -F ',' '{print $1, $2}' file.csv
```

In questo caso, il separatore è una virgola, quindi **awk** considererà i campi separati da virgole. Stampando **\$1**, **\$2**, otterrai il primo e secondo campo separati da una virgola.

4.2.3 Uso delle condizioni

`awk` consente anche di filtrare i dati in base a condizioni. Ad esempio, per stampare solo le righe in cui il primo campo è maggiore di 10:

```
1 awk '$1 > 10 {print $0}' file.txt
```

Questo comando stampa solo le righe in cui il primo campo è maggiore di 10.

4.2.4 Uso di Operatori e Aritmetica

`awk` permette di eseguire operazioni aritmetiche sui campi. Ad esempio, per sommare due campi:

```
1 awk '{print $1 + $2}' file.txt
```

Questo comando somma il primo e il secondo campo di ogni riga e stampa il risultato.

4.2.5 Modifica dell'output

Puoi anche modificare l'output utilizzando variabili interne come `OFS` (Output Field Separator) per separare i campi con un altro delimitatore. Per esempio, per separare i campi con una virgola:

```
1 awk 'BEGIN {OFS=","} {print $1, $2}' file.txt
```

In questo caso, l'output delle prime due colonne sarà separato da una virgola, anziché da uno spazio.

4.2.6 Eseguire operazioni su tutte le righe

Puoi anche definire operazioni che vengono eseguite su tutte le righe di un file. Ad esempio, per contare il numero di righe:

```
1 awk 'END {print NR}' file.txt
```

- `NR`: è una variabile predefinita in `awk` che tiene traccia del numero di righe elaborate.
- `END` è un blocco speciale che viene eseguito dopo che tutte le righe sono state processate.

4.2.7 Modificare il file In-Place

Mentre `sed` può modificare il file in-place con `-i`, `awk` non ha un'opzione nativa per fare modifiche in-place. Tuttavia, puoi reindirizzare l'output in un nuovo file e poi sostituirlo:

```
1 awk '{print $1, $2}' file.txt > nuovo_file.txt
```

4.2.8 Cicli e comandi complessi

`awk` supporta anche il controllo del flusso come cicli e condizioni. Ad esempio, puoi fare un ciclo per elaborare ogni campo:

```
1 awk '{for (i=1; i<=NF; i++) print $i}' file.txt
```

In questo caso, `NF` è una variabile che contiene il numero di campi per ogni riga, quindi questo comando stampa ogni campo di una riga.

5 Find

Il comando `find` in Bash è uno strumento molto potente e versatile per cercare file e directory all'interno di una struttura di directory, basato su vari criteri come nome, dimensione, data di modifica e altro. È particolarmente utile per trovare file in modo ricorsivo in tutte le sottocartelle di una directory, e può anche essere utilizzato per eseguire azioni sui file trovati, come spostarli o eliminarli.

La sintassi di base è:

```
1 find [percorso] [opzioni] [criteri] [azione]
```

- **percorso**: La directory in cui cercare (se omissso, `find` cercherà dalla directory corrente).
- **opzioni**: Opzioni per modificare il comportamento di `find` (ad esempio, per definire un'espressione booleana per i criteri).
- **criteri**: Condizioni per cercare i file (ad esempio, per nome, per dimensione, per data).
- **azione**: Azioni da eseguire sui file trovati (ad esempio, stampare il nome del file, eliminarlo, ecc.).

5.1 Criteri di ricerca comuni

5.1.1 Per nome (con `-name`)

Cerca file o directory con un determinato nome o pattern.

```
1 find /percorso -name "file.txt"
```

Questo comando cerca il file `file.txt` in tutto il percorso specificato.

5.1.2 Per tipo di file (con `-type`)

Cerca file di un tipo specifico. I tipi più comuni sono:

- `f` per file regolari.
- `d` per directory.
- `l` per link simbolici.

5.1.3 Per dimensione (con `-size`)

Cerca file di dimensione specifica. Puoi utilizzare varianti come `k` (kilobytes), `M` (megabytes), `G` (gigabytes). Esempio: Cercare file più grandi di 10 MB:

```
1 find /percorso -size +10M
```

5.1.4 Per data di modifica (con `-mtime` e `-atime`)

- `-mtime` cerca file modificati negli ultimi N giorni.
- `-atime` cerca file la cui data di accesso è stata modificata negli ultimi N giorni.

Esempio: Cercare file modificati negli ultimi 7 giorni:

```
1 find /percorso -mtime -7
```

5.1.5 Per permessi (con `-perm`)

Cerca file che hanno permessi specifici. Esempio: Cercare file con permessi di lettura e scrittura per il proprietario (644):

```
1 find /percorso -perm 644
```

5.1.6 Per proprietà (con `-user` e `-group`)

Cerca file di un determinato proprietario o gruppo. Esempio: Cercare file di proprietà dell'utente alice:

```
1 find /percorso -user alice
```

5.2 Azioni Comuni da Eseguire sui Risultati di `find`

5.2.1 Stampare i risultati (azione predefinita)

```
1 find /home/utente -name "*.log"
```

5.2.2 Eseguire un comando su ogni file trovato (con `-exec`)

Esempio, eliminare tutti i file `.log`:

```
1 find /home/utente -name "*.log" -exec rm {} \;
```

- `-exec`: permette di eseguire un comando su ogni file trovato.
- `{}`: viene sostituito con il nome del file trovato.
- `\;`: termina il comando `-exec`.

Esempio, per cambiare i permessi di tutti i file `.sh` trovati:

```
1 find /home/utente -name "*.sh" -exec chmod +x {} \;
```